

The Node.js fs module provides a comprehensive set of APIs for performing file I/O operations, allowing you to read, write, delete, and manipulate files and directories. Below is a detailed overview of the key functionalities available in the fs module, along with code examples for each operation.

1. Importing the fs Module

You can import the fs module using either CommonJS or ES6 syntax:

```
javascript
VerifyOpen In EditorRunCopy code
1// CommonJS
2const fs = require('fs');
3
4// ES6
5import * as fs from 'node:fs';
```

2. Reading Files

You can read files using both asynchronous and synchronous methods.

Asynchronous Reading

```
javascript
VerifyOpen In EditorRunCopy code
1fs.readFile('example.txt', 'utf8', (err, data) => {
2  if (err) {
3    console.error('Error reading file:', err);
4    return;
5  }
6  console.log('File content:', data);
7});
```

Synchronous Reading

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  const data = fs.readFileSync('example.txt', 'utf8');
3  console.log('Synchronous Read:', data);
4} catch (err) {
5  console.error('Error:', err);
6}
```

3. Writing Files

You can write to files using both asynchronous and synchronous methods.

Asynchronous Writing

```
javascript
VerifyOpen In EditorRunCopy code
1const content = "This is the content we're writing to the text file.";
2fs.writeFile('newFile.txt', content, 'utf8', (err) => {
3  if (err) {
4    console.error('Error writing file:', err);
5    return;
6  }
7  console.log('Text file written successfully');
```

```
8});
```

Synchronous Writing

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  fs.writeFileSync('syncFile.txt', content, 'utf8');
3  console.log('File written synchronously.');
```

4. Appending Data to Files

To append data to an existing file, use `fs.appendFile()`:

```
javascript
VerifyOpen In EditorRunCopy code
1const additionalContent = "\nAdditional content to append.";
2fs.appendFile('existingFile.txt', additionalContent, 'utf8', (err) => {
3  if (err) {
4    console.error('Error appending data:', err);
5    return;
6  }
7  console.log('Data appended to file.');
```

5. Deleting Files

You can delete files using `fs.unlink()` or `fs.unlinkSync()`:

Asynchronous Deletion

```
javascript
VerifyOpen In EditorRunCopy code
1fs.unlink('fileToDelete.txt', (err) => {
2  if (err) {
3    console.error('Error deleting file:', err);
4    return;
5  }
6  console.log('File deleted successfully.');
```

Synchronous Deletion

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  fs.unlinkSync('fileToDelete.txt');
```

6. Renaming Files

To rename a file, use `fs.rename()`:

```
javascript
```

```

VerifyOpen In EditorRunCopy code
1fs.rename('oldFileName.txt', 'newFileName.txt', (err) => {
2  if (err) {
3    console.error('Error renaming file:', err);
4    return;
5  }
6  console.log('File renamed successfully.');
```

7. Working with Directories

You can create, read, and delete directories using the following methods:

Creating a Directory

```

javascript
VerifyOpen In EditorRunCopy code
1fs.mkdir('newDirectory', { recursive: true }, (err) => {
2  if (err) {
3    console.error('Error creating directory:', err);
4    return;
5  }
6  console.log('Directory created successfully.');
```

Reading Directory Contents

```

javascript
VerifyOpen In EditorRunCopy code
1fs.readdir('directoryPath', (err, files) => {
2  if (err) {
3    console.error('Error reading directory:', err);
4    return;
5  }
6  console.log('Directory contents:', files);
7});
```

Deleting a Directory

```

javascript
VerifyOpen In EditorRunCopy code
1fs.rmdir('directoryToDelete', { recursive: true }, (err) => {
2  if (err) {
3    console.error('Error deleting directory:', err);
4    return;
5  }
6  console.log('Directory deleted successfully.');
```

8. Using Promises with the fs Module

The `fs/promises` API provides promise-based methods for file operations, which can be more convenient in modern JavaScript applications.

Example of Using Promises

```

javascript
VerifyOpen In EditorRunCopy code
1import { promises as fsPromises } from 'node:fs';
2
```

```

3async function fileOperations() {
4  try {
5    const data = await fsPromises.readFile('example.txt', 'utf8');
6    console.log('File content:', data);
7
8    await fsPromises.writeFile('newFile.txt', 'Hello World!', 'utf8');
9    console.log('File written successfully.');

```

Conclusion

The `fs` module in Node.js is a powerful tool for file I/O operations, providing both synchronous and asynchronous methods to handle files and directories efficiently. Whether you're reading, writing, or managing files, the `fs` module offers the flexibility needed for robust file handling in your applications.

To build an HTTP server using the Node.js `http` module, you can follow these steps. Below is a detailed explanation along with a complete code example.

Step-by-Step Guide to Building an HTTP Server in Node.js

1. Import the Required Modules:

- Use the built-in `http` module to create the server.
- Optionally, you can use the `fs` module to serve static files.

2. Create the Server:

- Use `http.createServer()` to create a server instance. This function takes a callback that handles incoming requests and sends responses.

3. Handle Requests:

- Inside the callback, you can check the request URL and method to determine how to respond.
- Use `res.writeHead()` to set the HTTP status code and headers.
- Use `res.end()` to send the response back to the client.

4. Listen on a Port:

- Call the `listen()` method on the server instance to start listening for incoming requests on a specified port.

Example Code

Here's a simple example of an HTTP server that responds with "Hello, World!" for any request:

javascript
[Verify](#)[Open In Editor](#)[Run](#)[Copy code](#)

```

1// Import the http module
2const http = require('http');
3
4// Define the port to listen on
5const port = process.env.PORT || 8000;
6
7// Create the server
8const server = http.createServer((req, res) => {
9    // Log the request URL
10    console.log(`Request received for '${req.url}'`);
11
12    // Set the response HTTP header with HTTP status and content type
13    res.writeHead(200, { 'Content-Type': 'text/plain' });
14
15    // Send the response body
16    res.end('Hello, World!\n');
17});
18
19// Start the server and listen on the specified port
20server.listen(port, () => {
21    console.log(`Server is running on http://localhost:${port}`);
22});

```

Explanation of the Code:

- **Importing the Module:** The `http` module is imported to create the server.
- **Defining the Port:** The server listens on port 8000 or any port specified in the environment variable `PORT`.
- **Creating the Server:** The `createServer` method takes a callback function that receives the request (`req`) and response (`res`) objects.
- **Logging Requests:** Each incoming request is logged to the console.
- **Setting Response Headers:** The `writeHead` method sets the HTTP status code (200 OK) and the content type (plain text).
- **Sending the Response:** The `end` method sends the response back to the client and closes the connection.
- **Listening for Requests:** The server starts listening on the specified port, and a message is logged to indicate that the server is running.

Running the Server

To run the server, save the code in a file named `server.js` and execute the following command in your terminal:

```

bash
VerifyOpen In EditorRunCopy code
1node server.js

```

You can then access the server by navigating to `http://localhost:8000` in your web browser, where you should see "Hello, World!" displayed.

Extending Functionality

You can extend this basic server to handle different routes, serve static files, or implement more complex logic by checking `req.url` and `req.method`. For example, you could serve an HTML file or respond with JSON data based on the request path.

If you have any specific requirements or need further assistance, feel free to ask!

To develop a Node.js web application using the `fs` (File System) and `http` modules, you can create a simple server that serves HTML content and handles file operations. Below, I will guide you through the steps to set up a basic web server that serves an HTML file and logs requests to a text file.

Step-by-Step Guide

1. Set Up Your Project Directory

- Create a new directory for your project:

```
bash
VerifyOpen In EditorRunCopy code
1mkdir node-web-app
2cd node-web-app
```

2. Create Your HTML File

- Create an `index.html` file that will be served by the server:

```
bash
VerifyOpen In EditorRunCopy code
1touch index.html
```

- Open `index.html` in a text editor and add some basic HTML content:

```
html
VerifyOpen In EditorRunCopy code
1<!DOCTYPE html>
2<html lang="en">
3<head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
6  <title>My Node.js Web App</title>
7</head>
8<body>
9  <h1>Welcome to My Node.js Web Application!</h1>
10  <p>This content is served from an HTML file.</p>
11</body>
12</html>
```

3. Create Your Node.js Server

- Create a new JavaScript file for your server:

```
bash
VerifyOpen In EditorRunCopy code
1touch server.js
```

- Open `server.js` in a text editor and add the following code:

```

javascript
VerifyOpen In EditorRunCopy code
1const http = require('http');
2const fs = require('fs').promises;
3const path = require('path');
4
5const host = 'localhost';
6const port = 8000;
7
8const requestListener = async (req, res) => {
9  try {
10    // Log the request to a file
11    await fs.appendFile('requests.log', `Requested: ${req.url} at $
{new Date().toISOString()}\n`);
12
13    // Serve the HTML file
14    if (req.url === '/') {
15      const filePath = path.join(__dirname, 'index.html');
16      const data = await fs.readFile(filePath);
17      res.writeHead(200, { 'Content-Type': 'text/html' });
18      res.end(data);
19    } else {
20      res.writeHead(404, { 'Content-Type': 'text/plain' });
21      res.end('404 Not Found');
22    }
23  } catch (error) {
24    console.error('Error handling request:', error);
25    res.writeHead(500, { 'Content-Type': 'text/plain' });
26    res.end('500 Internal Server Error');
27  }
28};
29
30const server = http.createServer(requestListener);
31
32server.listen(port, host, () => {
33  console.log(`Server is running on http://${host}:${port}`);
34});

```

4. Explanation of the Code

- **Modules Used:**
 - `http`: To create the server and handle HTTP requests.
 - `fs`: To read and write files asynchronously.
 - `path`: To handle file paths correctly across different operating systems.
- **Request Listener:**
 - The `requestListener` function handles incoming requests. It logs each request to a `requests.log` file and serves the `index.html` file when the root URL (`/`) is accessed.
 - If a user accesses any other URL, a 404 response is sent.

5. Run Your Server

- In your terminal, run the server using Node.js:

```
bash
```

```
VerifyOpen In EditorRunCopy code
1node server.js
```

- You should see the message indicating that the server is running.

6. Test Your Application

- Open your web browser and navigate to `http://localhost:8000`. You should see the HTML content displayed.
- Check the `requests.log` file to see the logged requests.

7. Stop the Server

- To stop the server, go back to your terminal and press `CTRL+C`.

Conclusion

You have successfully created a simple Node.js web application using the `fs` and `http` modules. This application serves an HTML file and logs requests to a file, demonstrating basic file handling and HTTP server functionality in Node.js. You can expand this application by adding more routes, serving different file types, or implementing additional features as needed.

Final Answer

Node.js Modules: fs and http

In this session, we will explore the `fs` (File System) and `http` modules in Node.js, focusing on file I/O operations, building an HTTP server, and developing a web application. Below is a detailed breakdown of each topic, including key points, subpoints, and examples to enhance your understanding.

1. Node.js fs Module: File I/O Operations

The `fs` module provides a set of APIs for performing file I/O operations, allowing you to read, write, delete, and manipulate files and directories.

1.1 Importing the fs Module

- **CommonJS Syntax:**

```
javascript
VerifyOpen In EditorRunCopy code
1const fs = require('fs');
```

- **ES6 Syntax:**

```
javascript
VerifyOpen In EditorRunCopy code
1import * as fs from 'node:fs';
```

1.2 Reading Files

- **Asynchronous Reading:**

```
javascript
VerifyOpen In EditorRunCopy code
1fs.readFile('example.txt', 'utf8', (err, data) => {
2  if (err) {
3    console.error('Error reading file:', err);
4    return;
5  }
6  console.log('File content:', data);
7});
```

- **Synchronous Reading:**

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  const data = fs.readFileSync('example.txt', 'utf8');
3  console.log('Synchronous Read:', data);
4} catch (err) {
5  console.error('Error:', err);
6}
```

1.3 Writing Files

- **Asynchronous Writing:**

```
javascript
VerifyOpen In EditorRunCopy code
1const content = "This is the content we're writing to the text file.";
2fs.writeFile('newFile.txt', content, 'utf8', (err) => {
3  if (err) {
4    console.error('Error writing file:', err);
5    return;
6  }
7  console.log('Text file written successfully');
8});
```

- **Synchronous Writing:**

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  fs.writeFileSync('syncFile.txt', content, 'utf8');
3  console.log('File written synchronously.');
```

```
4} catch (err) {
5  console.error('Error writing file synchronously:', err);
6}
```

1.4 Appending Data to Files

```
javascript
VerifyOpen In EditorRunCopy code
1const additionalContent = "\nAdditional content to append.";
2fs.appendFile('existingFile.txt', additionalContent, 'utf8', (err) => {
3  if (err) {
4    console.error('Error appending data:', err);
5    return;
6  }
7  console.log('Data appended to file.');
```

```
8});
```

1.5 Deleting Files

- **Asynchronous Deletion:**

```
javascript
VerifyOpen In EditorRunCopy code
1fs.unlink('fileToDelete.txt', (err) => {
2  if (err) {
3    console.error('Error deleting file:', err);
4    return;
5  }
6  console.log('File deleted successfully.');
```

- **Synchronous Deletion:**

```
javascript
VerifyOpen In EditorRunCopy code
1try {
2  fs.unlinkSync('fileToDelete.txt');
3  console.log('File deleted successfully.');
```

1.6 Renaming Files

```
javascript
VerifyOpen In EditorRunCopy code
1fs.rename('oldFileName.txt', 'newFileName.txt', (err) => {
2  if (err) {
3    console.error('Error renaming file:', err);
4    return;
5  }
6  console.log('File renamed successfully.');
```

1.7 Working with Directories

- **Creating a Directory:**

```
javascript
VerifyOpen In EditorRunCopy code
1fs.mkdir('newDirectory', { recursive: true }, (err) => {
2  if (err) {
3    console.error('Error creating directory:', err);
4    return;
5  }
6  console.log('Directory created successfully.');
```

- **Reading Directory Contents:**

```
javascript
VerifyOpen In EditorRunCopy code
1fs.readdir('directoryPath', (err, files) => {
2  if (err) {
3    console.error('Error reading directory:', err);
4    return;
5  }
```

```
6 console.log('Directory contents:', files);
7});
```

- **Deleting a Directory:**

```
javascript
VerifyOpen In EditorRunCopy code
1fs.rmdir('directoryToDelete', { recursive: true }, (err) => {
2  if (err) {
3    console.error('Error deleting directory:', err);
4    return;
5  }
6  console.log('Directory deleted successfully.');
```

1.8 Using Promises with the fs Module

```
javascript
VerifyOpen In EditorRunCopy code
1import { promises as fsPromises } from 'node:fs';
2
3async function fileOperations() {
4  try {
5    const data = await fsPromises.readFile('example.txt', 'utf8');
6    console.log('File content:', data);
7
8    await fsPromises.writeFile('newFile.txt', 'Hello World!', 'utf8');
9    console.log('File written successfully.');
```

2. Node.js http Module: Building an HTTP Server

The http module allows you to create an HTTP server that can handle requests and send responses.

2.1 Importing the Required Modules

```
javascript
VerifyOpen In EditorRunCopy code
1const http = require('http');
```

2.2 Creating the Server

```
javascript
VerifyOpen In EditorRunCopy code
1const server = http.createServer((req, res) => {
2  res.writeHead(200, { 'Content-Type': 'text/plain' });
3  res.end('Hello, World!\n');
4});
```

2.3 Listening on a Port

```
javascript
VerifyOpen In EditorRunCopy code
1const port = process.env.PORT || 8000;
2server.listen(port, () => {
3  console.log(`Server is running on http://localhost:${port}`);
4});
```

2.4 Complete Example

```
javascript
VerifyOpen In EditorRunCopy code
1const http = require('http');
2
3const port = process.env.PORT || 8000;
4
5const server = http.createServer((req, res) => {
6  console.log(`Request received for '${req.url}'`);
7  res.writeHead(200, { 'Content-Type': 'text/plain' });
8  res.end('Hello, World!\n');
9});
10
11server.listen(port, () => {
12  console.log(`Server is running on http://localhost:${port}`);
13});
```

3. Developing a Node.js Web Application Using fs and http Modules

You can create a simple web application that serves HTML content and logs requests.

3.1 Set Up Your Project Directory

```
bash
VerifyOpen In EditorRunCopy code
1mkdir node-web-app
2cd node-web-app
3touch index.html
```

3.2 Create Your HTML File

```
html
VerifyOpen In EditorRunCopy code
1<!DOCTYPE html>
2<html lang="en">
3<head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>My Node.js Web App</title>
7</head>
8<body>
9  <h1>Welcome to My Node.js Web Application!</h1>
10  <p>This content is served from an HTML file.</p>
11</body>
12</html>
```

3.3 Create Your Node.js Server

```
javascript
```

```

VerifyOpen In EditorRunCopy code
1const http = require('http');
2const fs = require('fs').promises;
3const path = require('path');
4
5const host = 'localhost';
6const port = 8000;
7
8const requestListener = async (req, res) => {
9  try {
10    await fs.appendFile('requests.log', `Requested: ${req.url} at ${new
Date().toISOString()}\n`);
11    if (req.url === '/') {
12      const filePath = path.join(__dirname, 'index.html');
13      const data = await fs.readFile(filePath);
14      res.writeHead(200, { 'Content-Type': 'text/html' });
15      res.end(data);
16    } else {
17      res.writeHead(404, { 'Content-Type': 'text/plain' });
18      res.end('404 Not Found');
19    }
20  } catch (error) {
21    console.error('Error handling request:', error);
22    res.writeHead(500, { 'Content-Type': 'text/plain' });
23    res.end('500 Internal Server Error');
24  }
25};
26
27const server = http.createServer(requestListener);
28
29server.listen(port, host, () => {
30  console.log(`Server is running on http://${host}:${port}`);
31});

```

Conclusion

In this session, we covered the `fs` module for file I/O operations and the `http` module for building an HTTP server. We also developed a simple web application that serves HTML content and logs requests. Understanding these modules is crucial for building robust Node.js applications that handle file operations and web requests effectively. If you have any further questions or need clarification on any topic, feel free to ask!